Midterm

Instructions

- PHY410: Do problems 1a-c and 2a-c
- PHY 505: Do problems 1a-d and 2a-d

Accept the assignment from "Classroom 50" website: https://classroom50.org/ubsuny/compphys-fall26/assignments/midterm/accept. This will create a new repository for you on github, titled something like `github.com/ubsuny/compphys-fall26-midterm-username`. The repository is located in the `ubsuny` github group, but it is your personal repository, and only you can view it. Technical detail: the repository is actually not a fork, but rather a brand new repository created by directly copying files from a "template repository."

Unlike the homework assignments, you will spend some time *generating your own data* for this midterm. We'll use phyphox, an iPhone/Android app that lets you access data from your phone's sensors. If you are unable to use the app (or if your phone does not have a barometer), please feel free to work in groups to generate the data, or ask the instructor as a last resort.

The assignment requires two uploads:

1. **Submit a writeup/lab report to UBLearns.** The writeup should contain your complete solutions to the problems in writing. Feel free to use any format: latex, markdown, etc. For assignments using Jupyter notebooks, you can also do your writeup inside the notebook using Markdown cells, export the notebook to HTML/PDF/etc. (this may require some additional setup), and upload the exported notebook.
2. **Upload your code through Classroom 50.** This just means to `git push` any code you wrote to your GitHub repository. You can push as many times as you like before the due date (in fact, this is a good idea, as it serves as a backup; nothing will be graded before the due date).

Extra credit

PHY410 students may optionally do 1d and/or 2d. The resulting score will replace the lowest homework problem grade (i.e. one 25 point problem), if higher, pro-rated if you only do 1d or 2d.

AI policy

You are allowed to use AI for code generation, but not for any text or data generation, and especially not for answering the full problem in one go. In your writeup (e.g., in an appendix or at the bottom of your Jupyter notebooks), describe any AI usage, include which LLM you used, and include the conversation log.

Problem 1: barometer vs. accelerometer

50-65 points

In this problem, we will use your phone's barometer and accelerometer to measure the motion of an elevator.

Prerequisites Download the phyphox app on your phone. Spend a minute getting familiar with the app:

- In a given "experiment," press "play" to start recording data and "pause" to stop recording.
- From a paused experiment, make sure you can export the data and transfer it to your computer, e.g., by email/Dropbox/your preferred method.

We'll use a custom experiment to collect acceleration and pressure simultaneously. You can either scan the QR code, or just hit the "+" icon and create a "simple experiment" with accelerometer (without g) and pressure data. Leave the time setting at "0" to specify maximum data rate.

Figure 1: https://dryu.web.cern.ch/dryu/public/elevator_a_p.phyphox

Problem 1a: data collection and visualization

10 points

Find an elevator, ideally with at least 5 floors (Cook/Hochstetter are the closest to Fronczak; note that floor 5 is inaccessible). Lay your phone flat on the ground with the screen up (consider using a paper towel, the floor may be dirty). **Using the accelerometer+pressure phyphox experiment, collect simultaneous barometer and accelerometer information versus time for 2x2=4 configurations**:

1. Up versus down
2. One trip versus stopping at each floor

Tips:

- I suggest taking a couple trials to ensure you have good data, but don't go too crazy and spend hours on this.
- Make sure you include about 5-10 seconds of "rest" data at the start and end of the elevator ride, where the elevator is stationary and the doors are not moving. This gives you good endpoint data for the exercise.
- Depending on your phone, the barometer and accelerometer will probably have different time granularity (the accelerometer has much higher sampling frequency). Make sure to use

the correct timestamps for each input data. Further, the time measurements will ideally be equally spaced; if not, you may have to use `np.interp` to convert to equal spacing.

Transfer the data to your computer. In the notebook `midterm/problem1/problem1.ipynb`, **visualize your data in one or more matplotlib plots**. The pressure and acceleration data should be plotted on similar time axes to allow for easy comparison. I also suggest overlaying some of your various experimental configurations and trials, to avoid having dozens of individual plots.

Problem 1b: data analysis (accelerometer)

20 points

Using a trapezoid integrator, **integrate the accelerometer data to obtain velocity** (v_{acc}), then **integrate again to obtain position** (z_{acc}). Identify the correct acceleration component and its sign: it's probably a_z, the direction perpendicular to your screen, but it may depend on your phone.

Note: in class, we covered integration with fixed limits; here, we are doing a cumulative integral, e.g., $v(t) = \int_0^t a(t')dt'$. The algorithm is the same, we are just reporting the results at each step of the integral. You might want to use `scipy.integrate.cumulative_trapezoid`.

Make a matplotlib plot showing the position, velocity, and acceleration on the same time axis (for example, a 3-panel plot stacked vertically).

One big problem with the accelerometer method is that sensor bias has a larger impact on the velocity and position. One easy way to see this is that the velocity will likely not be 0 where expected (e.g., when the elevator stops at floors). For each experiment, **correct your velocity data for this bias** with a simple method: draw a linear function between the velocities at the start and end of the experiment (which, again, we know should be 0), and subtract that line from the data. (You'll have to set a start and end time for this linear correction function; just estimate that as best you can.) Then, **integrate the corrected velocity** to obtain the corrected position. **Plot the results.**

Finally, **determine the floor spacing and total height** of your elevator ride. Include the standard deviation across your experiments.

Problem 1c: data analysis (barometer)

20 points

Perhaps surprisingly, the barometer in most phones is actually pretty good. Over small distances, the pressure and height have a linear relationship:

$$\rho_0 g \Delta h \approx -\Delta P$$

where $g = 9.81\,\text{m/s}^2$ and $\rho_0 \approx P_0 M/(RT)$ is the air density at the reference pressure P_0. Using:

- $R = 8.314\,\text{J/mol K}$
- $M = 0.02896\,\text{kg/mol}$
- $T = 22\,\text{C} = 295.15\,\text{K}$
- $P_0 = 1000\,\text{hPa}$

we have $\rho_0 \approx 1.180\,\text{kg/m}^3$. Optionally, feel free to correct this value for your measured P_0 and/or T, but the impact is small.

Use the formula to convert pressure (P) to height (z_P). Similarly to (1b), plot z_P vs. time, and determine the floor spacing and total height. Compare with the accelerometer results, and comment on any differences. Note, phyphox exports pressure in hPa=100 Pa.

Problem 1d: velocity from barometer

15 points

PHY505 students only

We can differentiate the barometric height (z_P) to obtain velocity. However, the data is quite noisy, so naively taking the derivative will give useless results. Instead, **use FFT to smooth the data** for one of your experiments. Remember the steps:

- The data have to be spaced uniformly. Check your barometer time data, and if not uniform, use `np.interp` to make it uniform.
- Prepare data for FFT: subtract the offset (or even better, subtract the line through the first and last points), pad the data to the nearest power of 2. Note: windowing may not work well here, as it can cause noise to blow up near the edges. If you do use a window, make sure to avoid that noise.
- Take the FFT
- Suppress the noise with a low-pass filter
- Take the inverse FFT
- Reverse the transformations you applied in step 1 (i.e. the window function and offset subtraction)

Then, take the numerical derivative of the smoothed data to obtain velocity. Plot it against the accelerometer results, and comment on any differences.

Problem 2: Doppler effect

50-60 points

In this problem, we provide a sound sample exhibiting the Doppler effect. To make life difficult, the audio sample is not just a single frequency, but rather contains a few frequencies plus some noise, emulating a train horn. We will determine the velocity of the train.

Assuming the train is traveling directly towards/away from the microphone, the formula for the Doppler equation is:

$$f(t) = \frac{f_0}{1 \pm \frac{v}{c}}.$$

where f_0 is the emitted frequency, v is the train speed, and $c = 343.4$m/s is the speed of sound. The frequency of the approaching train, $f_a = f_0/(1 - v/c)$, is higher, and the frequency of the receding train, $f_r = f_0/(1 + v/c)$, is lower.

Do your work in `midterm/problem2/problem2.ipynb`. The notebook demonstrates how to load the audio file into a numpy array. *You are allowed to use any FFT algorithm*, including `np.fft`, which is considerably faster than our in-class python function. Make sure to read the numpy documentation and understand the function.

Problem 2a: data visualization

10 points

Our tool for frequency analysis will be the Fourier transform. To get a feel for the data, **take the FFT and plot the power spectrum versus frequency**. Use a logarithmic y-axis. As usual, you should pad the data to the nearest power of 2 and apply a window function. You should also convert the spectral index to frequency. Comment on all the features you can see in the power spectrum.

Note: you may want to write a generic function for this task, as you will be performing a few more FFTs in this problem.

Problem 2b: frequency analysis aka spectrogram

20 points

The big difference from methods covered in lecture is that we want to analyze a changing frequency versus time. So, we have to break up the data into smaller intervals, each corresponding to a frequency measurement at a fixed time. This is called a *short time Fourier transform*. Specifically, we set a frame length, Δt, for the chunks. The Δt should be long enough for a good Fourier transform (i.e. contain enough cycles at the desired frequencies), but short enough that the frequency barely changes within one frame (and to avoid unnecessary computational time).

Write a function to perform the STFT. The function should accept an argument for the desired time, t, as well as the frame length, Δt. In other words, the STFT will perform an FFT on the data in the time window $[t - \Delta t/2,\ t + \Delta t/2]$.

Use the function to perform STFTs at a single representative t for the following window sizes: 0.02s, 0.1s, and 0.5s. Comment on any differences, and choose a single Δt value to use for the rest of the problem.

Finally, **evaluate and plot the STFT** for the full sample. You should evaluate it at t values with slightly smaller spacing than Δt, so that the frames overlap slightly between consecutive points (this gives a smooth result). The plot should have time on the x-axis, frequency on the y-axis, and the Fourier power on the z-axis (color scale), using `plt.pcolormesh` with a logarithmic z-axis. You should see the three peaks and the shift due to the Doppler effect.

Problem 2c: velocity extraction

20 points

For each of the 3 tones, **estimate the asymptotic frequency values**: the "approaching frequency", f_a, and the "receding frequency", f_r. Specifically, determine the average frequency for the first and last 2s of the sample. For **PHY410 students**: identifying the 3 maxima of the FFT power spectra is sufficient, using e.g. `np.argmax` in the vicinity of the peaks. For **PHY505 students**: to identify the frequency more precisely, please either (a) do parabolic interpolation of the peak, i.e., fit the log of the nearest 3 points with a parabola and find the maximum; or (b) aggressively zero-pad to $2^{17} = 131,072$ to obtain a finer frequency grid in your FFT.

Based on the extracted frequencies:

1. What is the velocity of the train? Report the mean and the spread of the values obtained from the 3 tones.
2. What is f_0, the emitted frequency? Reminder: this is the harmonic mean of f_a and f_r.

Problem 2d: non-radial motion

10 points

PHY505 students only

The Doppler formula used so far assumes the train is heading directly at the microphone. However, the sample was actually generated assuming that the microphone is placed a distance d away from the train tracks (which is, in general, a good life practice). Accounting for $d > 0$, the Doppler formula is:

$$f(t) = \frac{f_0}{1 + \frac{v}{c}\frac{x}{\sqrt{x^2+d^2}}}, \qquad x(t) \approx v(t - t_c)$$

where f_0 is the emitted frequency, v is the train speed, $c = 343.4$m/s is the speed of sound, d is the distance of closest approach, x is the train's position along the track, and t_c is the time of closest approach~[1].

We can correct for non-radial motion with a linear fit. For $|x| \gg d$, the Doppler formula is approximately:

$$f(t) \approx f_{\pm\infty} + \frac{K_\pm}{(t - t_c)^2}$$

Thus, we can use linear least-squares fitting to extract $f_{\pm\infty}$ (we don't care about $K_\pm$). Specifically:

[1] Note: more precisely, we should account for the sound travel time. For example, the time of closest approach, t_0, is not exactly the same as the time of measuring the sound emitted at t_0: $t_c = t_0 + d/c$. For this problem, however, this is a tiny correction.

1. Estimate t_c, the time at which $f(t) = (f_a + f_r)/2$.
2. For the first and last 2s of the data, perform linear fit of $f(t)$ vs. $1/(t - t_c)^2$, and record the constant term, $f_{\pm\infty}$.
3. Using $f_{\pm\infty}$, what is v? How does it compare to the answer from (2c)?